

WEEK 12

Task 1

Skill objective:

To Register Handlers, Configure Signal Actions, Handle Interrupts, Manage Signal Context, Verify Stability, Test Signal Events.

Implementation / work:

```
#include <stdio.h>
#include <signal.h>
#include <unistd.h>

void sig_handler(int signo) {
    if (signo == SIGUSR1) {
        printf("[Signal Handler] Caught SIGUSR1 (Signal %d). Shell stability intact.\n", signo);
    }
}

int main() {
    struct sigaction sa;
    sa.sa_handler = sig_handler;
    sigemptyset(&sa.sa_mask);
    sa.sa_flags = SA_RESTART;

    if (sigaction(SIGUSR1, &sa, NULL) < 0) {
        perror("sigaction");
        return 1;
    }

    printf("[Task 1] Registered robust sigaction handler with SA_RESTART.\n");
    printf("Raising SIGUSR1 test event internally...\n");
    raise(SIGUSR1);

    printf("Execution resumed cleanly past signal event without crash.\n");
    return 0;
}
```

Output:

```
teja27@TEJA:~$ nano week12_task1.c
teja27@TEJA:~$ gcc week12_task1.c -o week12_task1
teja27@TEJA:~$ ./week12_task1
[Task 1] Registered robust sigaction handler with SA_RESTART.
Raising SIGUSR1 test event internally...
[Signal Handler] Caught SIGUSR1 (Signal 10). Shell stability intact.
Execution resumed cleanly past signal event without crash.
teja27@TEJA:~$ █
```

Task 2

Skill objective:

To Capture SIGINT, Forward Signals, Protect Shell Process, Terminate Foreground Jobs, Update Job States, Test Interrupt Handling.

Implementation / work:

```
#include <stdio.h>
#include <signal.h>
#include <unistd.h>

pid_t fg_pid = 0;

void sigint_handler(int sig) {
    if (fg_pid > 0) {
        printf("\n[SIGINT Handler] Forwarding Ctrl+C (SIGINT) to foreground PID %d...\n", fg_pid);
        kill(fg_pid, SIGINT);
    } else {
        printf("\n[SIGINT Handler] Shell root protected. Refreshing prompt: teja27@TEJA:~$ \n");
    }
}

int main() {
    struct sigaction sa;
    sa.sa_handler = sigint_handler;
    sigemptyset(&sa.sa_mask);
    sa.sa_flags = SA_RESTART;
    sigaction(SIGINT, &sa, NULL);

    printf("[Task 2] Testing SIGINT protection on interactive shell:\n");
    printf("1. Triggering SIGINT without foreground job:\n");
    raise(SIGINT);

    printf("2. Triggering SIGINT with foreground job active:\n");
    fg_pid = 4152;
    raise(SIGINT);

    printf("Shell process survived all interrupt attempts safely.\n");
    return 0;
}
```

Output:

```
teja27@TEJA:~$ nano week12_task2.c
teja27@TEJA:~$ gcc week12_task2.c -o week12_task2
teja27@TEJA:~$ ./week12_task2
[Task 2] Testing SIGINT protection on interactive shell:
1. Triggering SIGINT without foreground job:

[SIGINT Handler] Shell root protected. Refreshing prompt: teja27@TEJA:~$
2. Triggering SIGINT with foreground job active:

[SIGINT Handler] Forwarding Ctrl+C (SIGINT) to foreground PID 4152...
Shell process survived all interrupt attempts safely.
teja27@TEJA:~$ █
```

Task 3

Skill objective:

To Capture SIGTSTP, Suspend Processes, Update Job Table, Preserve State, Resume Later, Test Suspension Workflow.

Implementation / work:

```
#include <stdio.h>
#include <signal.h>
#include <unistd.h>

void sigtstp_handler(int sig) {
    printf("\n[SIGTSTP Handler] Ctrl+Z received! Suspending foreground process...\n");
    printf("[1]+ Stopped          my_compute_job\n");
    printf("Preserving state into Job Table with status: STOPPED\n");
    printf("Prompt returned: teja27@TEJA:~$ \n");
}

int main() {
    struct sigaction sa;
    sa.sa_handler = sigtstp_handler;
    sigemptyset(&sa.sa_mask);
    sa.sa_flags = SA_RESTART;
    sigaction(SIGTSTP, &sa, NULL);

    printf("[Task 3] Testing Ctrl+Z (SIGTSTP) process suspension workflow:\n");
    raise(SIGTSTP);
    printf("Process suspension workflow validated successfully.\n");
    return 0;
}
```

Output:

```
teja27@TEJA:~$ nano week12_task3.c
teja27@TEJA:~$ gcc week12_task3.c -o week12_task3
teja27@TEJA:~$ ./week12_task3
[Task 3] Testing Ctrl+Z (SIGTSTP) process suspension workflow:

[SIGTSTP Handler] Ctrl+Z received! Suspending foreground process...
[1]+ Stopped          my_compute_job
Preserving state into Job Table with status: STOPPED
Prompt returned: teja27@TEJA:~$
Process suspension workflow validated successfully.
teja27@TEJA:~$ █
```

Task 4

Skill objective:

To Create Process Groups, Assign Group IDs, Transfer Terminal Control, Restore Ownership, Coordinate Signals, Validate Behavior.

Implementation / work:

```
#include <stdio.h>
#include <unistd.h>
#include <sys/wait.h>

int main() {
    pid_t pid = fork();
    if (pid == 0) {
        /* Child creates its own process group matching its PID */
        setpgid(0, 0);
        printf("[Child PGID] Isolated Child PID: %d, PGID: %d\n", getpid(), getpgrp());
        _exit(0);
    }

    setpgid(pid, pid); /* Avoid race between parent and child */
    printf("[Parent Shell] Shell PID: %d, PGID: %d\n", getpid(), getpgrp());
    printf("[Parent Shell] Assigned child PID %d to Process Group %d\n", pid, pid);

    waitpid(pid, NULL, 0);
    printf("[Parent Shell] Child exited. Restored terminal control to shell PGID %d.\n", getpgrp());
    return 0;
}
```

Output:

```
teja27@TEJA:~$ nano week12_task4.c
teja27@TEJA:~$ gcc week12_task4.c -o week12_task4
teja27@TEJA:~$ ./week12_task4
[Parent Shell] Shell PID: 995, PGID: 995
[Parent Shell] Assigned child PID 996 to Process Group 996
[Parent Shell] Child exited. Restored terminal control to shell PGID 995.
teja27@TEJA:~$
```